



Introduction to R and RStudio

Qondeni Ndlangamandla

South African Medical Research Council (SAMRC)

Introduction to R: Pre-Workshop Course



Installing R and RStudio

Step 1: Install R

- ▶ Visit <https://cran.r-project.org>
- ▶ Select your operating system:
 - ▶ Windows
 - ▶ macOS
 - ▶ Linux
- ▶ Download and run the installer.

Step 2: Install RStudio

- ▶ Visit <https://posit.co/download/rstudio-desktop/>
- ▶ Download **RStudio Desktop (Free)**.
- ▶ Install RStudio after R has been installed.

Step 3: Verify the Installation

- ▶ Open RStudio.
- ▶ Check that the **R Console** appears.
- ▶ Create a new project and start coding.

Ready to Code?

Once RStudio is open and the R Console is visible, you are ready to start coding in R.

What is R?

Definition

R is a free, open-source programming language designed for data analysis, statistics, and data visualization.

Key Features

- ▶ Free and open-source software.
- ▶ Powerful tools for data cleaning, analysis, and modelling.
- ▶ Extensive package ecosystem, including `dplyr`, `ggplot2`, and `readr`.
- ▶ Produces high-quality data visualizations.
- ▶ Supports reproducible research and reporting.
- ▶ Large and active global user community.

What is RStudio?

Definition

RStudio is an Integrated Development Environment (IDE) that makes it easier to write, run, and manage R code.

Key Features

- ▶ User-friendly interface for coding and data analysis.
- ▶ Integrated script editor, console, and workspace.
- ▶ Built-in tools for plotting, package management, and debugging.
- ▶ Supports project management and reproducible workflows.
- ▶ Seamless integration with R Markdown for reporting.

R vs RStudio

- ▶ **R** is the *programming language*, while **RStudio** is the *environment* used to interact with R.

Think of a car:

- ▶ **R** is the **engine** that does the actual work, such as:
 - ▶ calculations,
 - ▶ statistical analyses, and
 - ▶ creating graphs.
- ▶ **RStudio** is the **dashboard and controls** that make it easier to drive the car. It provides a user-friendly interface for:
 - ▶ writing code,
 - ▶ viewing results,
 - ▶ managing files, and
 - ▶ creating plots.

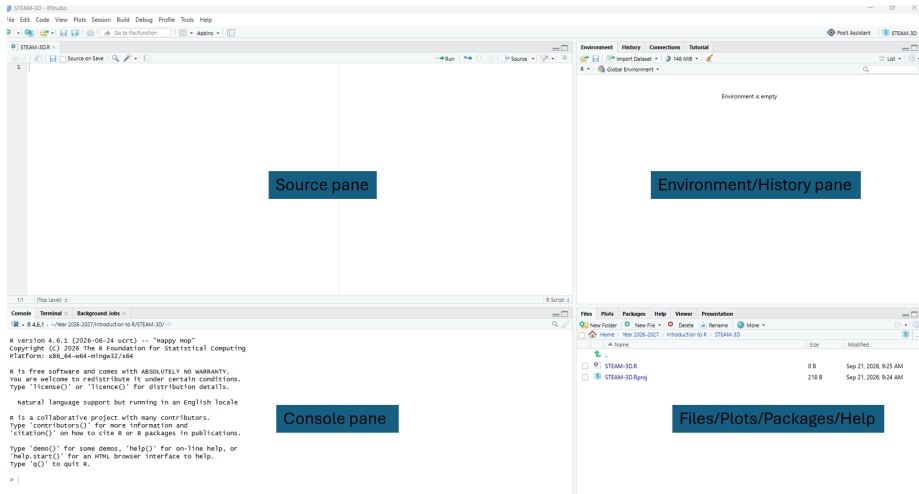
R (Engine) $\longrightarrow$ **RStudio (Dashboard/Interface)**

RStudio Interface Overview

Main Components:

- ▶ **Source Pane:** Write and edit R scripts.
- ▶ **Console Pane:** Execute R commands and view outputs.
- ▶ **Environment/History Pane:** Displays variables and command history.
- ▶ **Files/Plots/Packages/Help Pane:** Provides file navigation, plot viewing, package management, and access to help.

RStudio Interface Overview



The Source, Console, Environment/History, and Files/Plots/Packages/help Panes.

Source Pane

Purpose: Write, edit, and save R scripts.

Key Features:

- ▶ Syntax highlighting.
- ▶ Code completion and suggestions.
- ▶ Run selected lines or entire scripts.
- ▶ Easy script organization and editing.

Console Pane

Purpose: Execute R commands interactively.

Key Features:

- ▶ Immediate feedback.
- ▶ Command history with arrow keys.
- ▶ Quick calculations and testing.
- ▶ Displays errors and warnings.

Environment and History Panes

Environment Tab:

- ▶ Displays all current variables and/or objects.
- ▶ Shows object names, types, and contents.

History Tab:

- ▶ Stores previously executed commands.
- ▶ Commands can be reused, edited, and rerun.

Files, Plots, Packages, and Help

- ▶ **Files:** Navigate and open files.
- ▶ **Plots:** View and export visualizations.
- ▶ **Packages:** Install and manage R packages.
- ▶ **Help:** Access R documentation and help files.

R Projects

What is an R Project?

- ▶ An R Project is a self-contained workspace for a specific analysis or study.
- ▶ Stores scripts, data, outputs, and project settings in one location.

Why Use Projects?

- ▶ Organizes files related to a specific analysis.
- ▶ Automatically sets the working directory to the project folder.
- ▶ Makes code more reproducible and portable.
- ▶ Reduces errors caused by incorrect file paths.
- ▶ Simplifies collaboration and project management.

Working with R Projects

Why Not Use `setwd()`?

What is `setwd()`?

`setwd()` changes the **working directory** in R.

Example:

```
setwd("C:/MyProject")
```

- ▶ Paths may differ across computers.
- ▶ Scripts may fail when folders are moved.
- ▶ Makes sharing code more difficult.

How to Create a Project

1. Go to **File > New Project**.
2. Choose:
 - ▶ New Directory,
 - ▶ Existing Directory, or
 - ▶ Version Control.
3. Select a location and project name.

Best Practice

Use an **R Project** instead of manually setting the working directory with `setwd()`.

Saving and Managing R Scripts

Saving Scripts

- ▶ Save scripts using **File > Save** or **Ctrl + S**.
- ▶ Save scripts within your **R Project folder**.
- ▶ Save frequently to avoid losing work.

Managing Scripts

- ▶ Use meaningful and descriptive file names.
- ▶ Organize scripts into folders.
- ▶ Add comments to document your code.
- ▶ Keep scripts focused on a specific task or analysis.

Managing Scripts: Best Practice Example

Instead of:

- ▶ `script1.R`
- ▶ `mycode.R`
- ▶ `test.R`

Use:

- ▶ `01_data_import.R`
- ▶ `02_data_cleaning.R`
- ▶ `03_analysis.R`
- ▶ `04_visualization.R`

Running Code and Viewing Results

Running Code

- ▶ **Run selected lines:** Ctrl + Enter.
- ▶ **Run the entire script:** Use the **Source** button in the Source Pane.
- ▶ Execute commands directly in the **Console Pane**.

Viewing Results

- ▶ Text output appears in the **Console Pane**.
- ▶ Graphs appear in the **Plots Pane**.
- ▶ Errors and warnings help identify and fix code errors.

R Markdown in RStudio

What is R Markdown?

- ▶ A tool for creating **dynamic and reproducible reports**.
- ▶ Combines:
 - ▶ text,
 - ▶ R code,
 - ▶ results, and
 - ▶ visualizations.
- ▶ Integrates all content into a **single document**.

Key Features

- ▶ Integrates analysis and reporting.
- ▶ Generates outputs such as:
 - ▶ HTML,
 - ▶ PDF, and
 - ▶ Word documents.
- ▶ Regenerates results and outputs when the document is rendered.
- ▶ Supports reproducible research and collaboration.

R Functions, Packages and Libraries.

Functions in R

What is a Function?

- ▶ A function is a **reusable block of code** that performs a specific task.
- ▶ Functions can accept inputs, called **arguments**, and return outputs.
- ▶ Examples of built-in functions:
 - ▶ `mean(c(2, 4, 6, 8))`
 - ▶ `sum(c(1, 2, 3))`
 - ▶ `sqrt(25)`

Why Use Functions?

- ▶ Reduce repetitive coding.
- ▶ Improve code efficiency and readability.
- ▶ Perform calculations and analyses consistently.
- ▶ Make code easier to reuse and maintain.

R Packages

What is a Package?

- ▶ A package is a collection of **functions, datasets, and documentation**.
- ▶ Packages extend the functionality of **base R**.

Why Use Packages?

- ▶ Perform specialized analyses.
- ▶ Create advanced visualizations.
- ▶ Import and manipulate data efficiently.

Installing a Package

Installation

```
install.packages("package_name")
```

Examples of Popular Packages

- ▶ ggplot2 – Data visualization.
- ▶ dplyr – Data manipulation.
- ▶ readr – Data import.

Loading R Packages with `library()`

What is a Library?

- ▶ `library()` makes an installed package available for use in the current R session.
- ▶ Packages must be **loaded** before their functions can be used.

Loading a Package:

Example

```
library(ggplot2)
```

How It Works

1. **Install** the package once.
2. **Load** the package using `library()`.
3. **Use** the package's functions.

```
install.packages("ggplot2")
```



```
library(ggplot2)
```



```
ggplot(...)
```

R Projects, Data types and Data Structures .

R Objects, Data Types, and Data Structures

How Does R Store Data?

R Objects

- ▶ Everything in R is stored as an object.
- ▶ Each object has:
 1. **A Data type** – describes the kind of value being stored.

Examples:

- ▶ Number
- ▶ Text
- ▶ TRUE/FALSE

2. A Data Structure

textbf Describes how the values are organized.

Examples:

- ▶ Scalar
- ▶ Vector
- ▶ List

Data Types in R

Common data types:

Data Type	Description
Numeric	Numbers
Character	Text
Logical	TRUE or FALSE
Factor	Categories

Data Type describes the kind of value being stored.

Data Types Examples: R language mode

Numeric

```
X <- 25
```

Character

```
Colour <- "Red"
```

Factor

```
Gender <- factor(c("Male",  
"Female"))
```

Logical

```
Smoker <- TRUE
```

R language mode uses different data types to represent different kinds of values.

The Numerical, Character, Logical and Factor Data Types.

Numeric Data Type

Used to store numbers

Examples:

```
Age <- 31  
Height <- 1.89
```

```
Income <- 11600  
Population_size <- 220000
```

Numeric = Numbers

Character data Type

Used to store text

Examples:

```
Province <- "KwaZulu-Natal"  
Colour <- "Red"
```

Characteristics:

- ▶ Stored inside quotation marks
- ▶ Cannot be used for arithmetic calculations

Character = Text

Logical Data Types

Stores only two possible values (TRUE/FALSE)

Examples:

Vaccinated

```
Vaccinated <- TRUE
```

Smoker

```
Smoker <- FALSE
```

Logical = TRUE / FALSE

Factor data Type

Used for categorical variables

Examples:

```
Province <- factor(c("KZN", "GP", "WC", "EC"))  
Marital_status <- factor(c("Single", "Married", "Widow",  
"Divorced"))
```

The elements stored inside the factor variable are referred to as levels.

Data Structures in R

What are data structures?

A data structure determines how data are organized within an object.

Common data structures:

1. **Scalar**
2. **Vector**

3. **List**
4. **Data frame**

Scalar • Vector • List • Data frame

Scalar

A scalar contains a single value

Example:

```
age <- 31
```

```
Name <- "Joh"
```

```
Smoker <- TRUE
```

Key feature: One object = one value

Vector

A vector is a collection of values of the same data type.

Numerical vector:

```
ages <- c(20, 25, 31, 35)
```

Character vector:

```
Provinces <- c("GP", "KZN", "WC",  
               "EC")
```

Key feature: All values must be the same type

List

A list can contain different types of objects.

Example:

```
participant <- list(ID = 101, Colour = "Red", Age = 35, Smoker =  
                    FALSE)
```

Key feature

It can mix data types and data structures

Think of a list as a folder

Think of a list as a folder containing different files

Data frame

A data frame stores data in rows and columns.

Example:

```
patients <- data.frame(ID = c(1,2,3), Age = c(25,30,45), Sex =  
                        c("M","F","F"))
```

Key feature

Each column can have a different data type.

Think of a data frame as an excel file

Think of a data frame as an Excel file spreadsheet in R

Summary (Data Types vs Data Structures)

Data Types (What is stored?)

- ▶ Numeric
- ▶ Character
- ▶ Logical
- ▶ Factor

Data Structures (How it is stored?)

- ▶ Scalar
- ▶ Vector
- ▶ List
- ▶ Data Frame

What is stored?



How is it stored?

Rtools

What are Rtools?

RTools is a collection of tools and compilers that enables R to build and install packages from source code on Windows.

Rtools works behind the scenes when R needs to build packages.

Why is RTools Important?

Why is RTools Important?

Rtools allows users to:

01

Install packages that contain compiled code

02

Build R packages from source

03

Develop and test custom R packages

04

Improve compatibility with package updates

Rtools allows users to work with packages that require compiled code.

Rtools: Key Takeaway

Key takeaway:

RTools acts as the bridge between R and low-level programming languages (C, C++, and Fortran), allowing advanced packages to be compiled and installed successfully.

Rtools: <https://cran.r-project.org/bin/windows/Rtools/>

Conclusion

- ▶ **R** is a powerful language for statistical analysis, data science, and visualization.
- ▶ **RStudio** provides an efficient environment for coding and project management.
- ▶ **Projects and scripts** promote organized and reproducible workflows.
- ▶ **R Markdown** enables integrated analysis and reporting.
- ▶ **Functions and packages** extend what you can do with R.
- ▶ Continue exploring **packages, documentation, and community resources**.

Keep practising, exploring, and learning R!